Software Requirements Specification

SchEDU

Adaptive Academic Pacing Using Deterministic Constraint-Based Greedy Scheduling and Heuristic Rebalancing

Prepared by

Aundreka Clarinze Perez
Marjinel Abante
Leo Franklin Borromeo

College of Information Technology and Computer Science
Lyceum of the Philippines University – Cavite

Software Engineering 2

May 14, 2026

Contents

Document Control

Document Title	Software Requirements Specification for SchEDU
System Name	SchEDU: Adaptive Academic Pacing Using Deterministic Constraint-Based Greedy Scheduling and Heuristic Rebalancing
Prepared By	Aundreka Clarinze Perez, Marjinel Abante, and Leo Franklin Borromeo
Institution	College of Information Technology and Computer Science, Lyceum of the Philippines University – Cavite
Course Context	Software Engineering 2
Version	Rev. 2
Date	May 14, 2026
Implementation Status	Defense prototype with implemented mobile workflows, Supabase-backed persistence, repository-local scheduling logic, and auxiliary document services.

Revision History

Revision	Date	Description
Rev. 1	May 14, 2026	Initial repository-aligned SRS covering implemented SchEDU features and scheduler behavior.
Rev. 2	May 14, 2026	Added formal title page, separated table of contents, strengthened defense-ready implementation scope, clarified live-demo assumptions, added acceptance criteria and traceability evidence, and tightened requirement wording.

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, non-functional, interface, data, validation, and demonstration requirements of SchEDU. The document is intended to serve as the shared specification for implementation, evaluation, testing, documentation, and academic defense of the current system. It describes the implemented defense prototype rather than an abstract future proposal, while still identifying boundaries that are not claimed by the current repository.

1.2 Product Scope

SchEDU is a mobile academic planning system for teachers. It consolidates curriculum organization, lesson planning, deterministic schedule generation, calendar maintenance, curriculum text extraction, and AI-assisted activity generation within a single application. The system is designed around teacher-level instructional planning rather than institution-wide room allocation, student enrollment management, or global campus timetabling.

For defense purposes, SchEDU is treated as a full implemented system that can be demonstrated live through authenticated mobile workflows. A successful demo should show a teacher moving from account access and academic setup to subject content, lesson-plan generation, calendar inspection, disruption handling, and document-support workflows using repository-backed code and Supabase-backed data.

The current system supports the following major capabilities:

- authenticated user access with profile and theme management;
- institution, section, and subject management;
- structured storage of units, chapters, lessons, and lesson-plan scope;
- deterministic rule-based schedule generation, compression, and rebalance;
- monthly and daily calendar views with block-level maintenance actions; and
- document support workflows for curriculum extraction and activity generation.

1.3 Intended Audience

This document is intended for the following stakeholders:

- project proponents and panel members who need a precise system definition;
- developers implementing or extending the application;
- testers validating requirements against system behavior;
- teachers and pilot users reviewing supported workflows; and
- administrators evaluating operational and reporting capabilities.

1.4 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification.
OAuth	Third-party authentication flow such as Google, Apple, or Facebook sign-in.
OCR	Optical Character Recognition used to extract text from images.
RLS	Row-Level Security used by Supabase/PostgreSQL to restrict data access.
Lesson Plan	A teacher-owned planning record that binds a subject, section, date range, and selected content to generated schedule state.
Slot	A schedulable class meeting occurrence derived from recurring meeting patterns and date ranges.
Block	A scheduled instructional or assessment unit such as a lesson, written work, performance task, exam, or buffer.
Blackout	A date or slot made unavailable by events, delays, suspension, leave, or related constraints.
PT	Performance Task.
WW	Written Work.

1.5 Document Overview

Section 2 describes the product context, actors, operating environment, constraints, and live-demo assumptions. Section 3 defines the external interfaces. Section 4 defines the core data requirements. Section 5 lists the functional requirements by feature area. Section 6 defines non-functional requirements. Section 7 defines acceptance criteria and traceability evidence. Section 8 records the current system boundaries to prevent overclaiming unsupported behavior.

2 Overall Description

2.1 Product Perspective

SchEDU is a client-server mobile application built with React Native and Expo Router on the client side and Supabase services on the backend. The client contains the primary planning workflows, while the backend manages user access, structured academic records, stored files, and server-side utility functions.

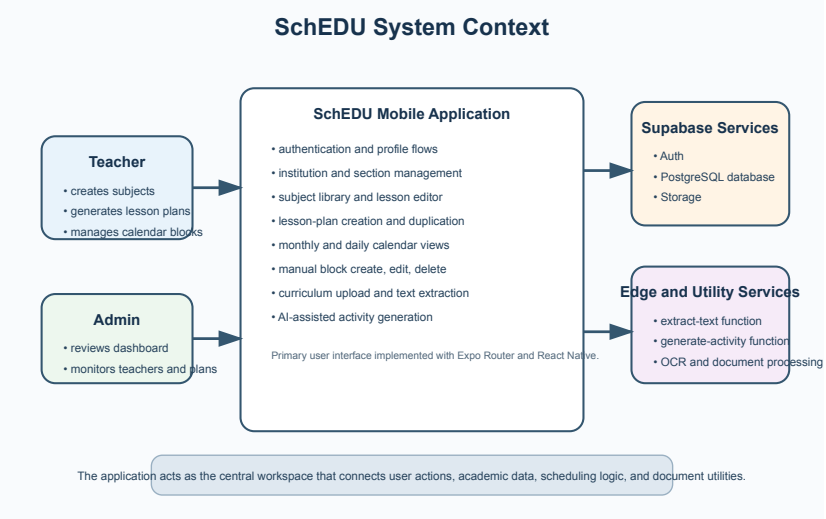


Figure 1: SchEDU system context

As shown in Figure 1, teachers and administrators use the same application context but interact with different feature subsets. The mobile application acts as the central workspace that coordinates persistent academic data, scheduling logic, and supporting utility services.

2.2 Product Functions

At a high level, the system provides the following functions:

- authenticate users and maintain profile, theme, and session state;
- manage institutions, sections, subjects, units, chapters, and lessons;
- create lesson plans using selected content, date ranges, and meeting patterns;
- generate schedule slots and blocks using a deterministic constraint-based greedy scheduler;
- display lesson-plan outputs in plan, monthly calendar, and daily agenda views;
- allow block-level maintenance such as create, edit, delete, suspend, and refresh actions;
- extract text from uploaded curriculum documents; and
- generate and export classroom activity drafts in PDF and DOCX formats.

2.3 Scheduling Methodology

The scheduler is implemented as a deterministic constructive pipeline rather than as a global search or metaheuristic procedure. A single run consumes the lesson-plan record, recurring meeting patterns, selected lesson scope, stored activity requirements, academic-term rules, blackout events, delay records, and any existing slot or block state that must be preserved. The scheduler emits runtime slots, runtime blocks, per-term schedule-plan metadata, ordered slot output, warnings, and summary metrics.

The methodology proceeds in the following stages:

1. **Slot normalization and expansion.** Meeting patterns are normalized by weekday and time, assigned stable slot numbers and series keys, validated so that overlapping same-day instances are rejected, and expanded across the lesson-plan date range into dated runtime slots.
2. **Blackout mapping and slot-state merge.** School calendar events and teacher delays are mapped onto slot dates as blackout information. Existing slot rows may be merged into generated slots so that persistent identifiers and manual lock state can survive regeneration where applicable.
3. **Term decomposition and block derivation.** Usable slots are grouped into academic terms ending on configured exam dates. Lessons, written work, and performance-task counts are distributed across terms by deterministic equal baseline allocation with earlier-term remainder priority, quiz blocks are derived from lesson dependencies, and each term receives an exam block and its exam-slot anchor. If no normal class slot exists on the exam date, a synthetic exam slot is created for that term.
4. **Constraint-based greedy placement.** The scheduler reserves exam slots first, then greedily assigns lesson blocks to the next usable chronological content slot. Due quizzes are released when all prerequisite lessons have been placed, while written work and performance tasks are inserted according to computed term intervals. Placement is deterministic because the next feasible slot and the next due block are always selected by fixed ordering rules rather than by randomized search.
5. **Heuristic compression, ordering, and rebalance.** When usable slot capacity is lower than the number of remaining content blocks, assessments may be compressed onto compatible lesson slots according to bounded heuristics. Non-quiz written work prefers a lesson slot already in use, while performance tasks first prefer a lesson slot that does not already contain written work. After placement, within-slot order is normalized for display stability. When availability changes later, non-pinned blocks are released and re-flowed while teacher-locked blocks, exam blocks, and protected buffers remain preserved where possible.

The methodology depends on one important implementation invariant: teacher actions that must survive future rebalancing are represented as locked schedule state. A manually pinned block that later becomes infeasible is surfaced as a visible conflict for re-pinning rather than being silently relocated by the scheduler.

2.4 Live Demonstration Profile

The system is expected to support a live defense demonstration using a configured Expo/React Native build and a reachable Supabase project. The demonstration should be prepared with sample academic data so that reviewers can observe the implemented end-to-end behavior without waiting for manual data entry for every record.

Demo Area	Expected Demonstration	Evidence of Completion

Authentication and profile	Sign in, load the authenticated user context, and open profile or settings screens.	Protected routes load only after authentication and user metadata is displayed.
Academic setup	Open institution, section, subject, chapter, and lesson workflows.	Teacher-owned academic records appear in library and planning screens.
Lesson-plan generation	Select a subject, section, date range, meeting pattern, term dates, and lesson scope, then generate the plan.	Plan, slots, selected content, and blocks are persisted and visible in plan/calendar views.
Calendar maintenance	Open monthly and daily views, inspect blocks, create or edit a block, and suspend or restore a teaching day.	Calendar refreshes, blocks remain ordered, and schedule maintenance actions preserve data.
Rebalance and conflict handling	Trigger a blackout or suspension affecting a plan with scheduled blocks.	Non-locked blocks re-flow, locked conflicts remain visible, and no scheduled block is silently discarded.
Document support	Upload a curriculum source or invoke activity generation for written work or performance tasks.	Extracted or generated text is returned in an editable form and can be exported where configured.
Administrative overview	Open admin dashboard screens using an authorized role.	Teacher, subject, lesson-plan, and activity summaries render from backend records.

2.5 Operational Prerequisites for Defense

The following conditions must be satisfied before a live demonstration:

- a local or device build must have the required Expo/React Native runtime available;
- `EXPO_PUBLIC_SUPABASE_URL` and `EXPO_PUBLIC_SUPABASE_KEY` must point to a provisioned Supabase project;
- the database schema and RLS policies in the repository setup scripts must be applied;
- demo users, school records, sections, subjects, and lessons should be prepared in advance;
- the `extract-text` and `generate-activity` Edge Functions must be deployed for utility demonstrations;
- image OCR should be demonstrated in a development build, not Expo Go, because

the native OCR dependency requires native binaries; and
- backup screenshots or prepared data should be available in case external network or AI service availability affects utility workflows.

2.6 User Classes and Characteristics

User Class	Description	Skill Level	Typical Responsibilities
Teacher	Primary end user who manages instructional content and schedules.	Moderate	Create subjects, lessons, activities, lesson plans, and maintain calendar state.
Admin	Administrative user who monitors teacher, subject, and lesson-plan summaries.	Moderate	Review dashboard metrics, recent activity, teacher counts, and plan completion indicators.
Superadmin	Elevated administrative role represented in the data model.	High	Perform the responsibilities of an admin with higher institutional authority where configured.

2.7 Operating Environment

- **Client platform:** React Native application built with Expo Router.
- **Primary targets:** Android and iOS mobile devices.
- **Backend platform:** Supabase Auth, PostgreSQL, Storage, and Edge Functions.
- **Development environment:** Node.js, TypeScript, Expo toolchain, ESLint, and Git-based version control.
- **Document utilities:** PDF and DOCX generation libraries, file upload support, and OCR/document extraction services.
- **Defense validation environment:** local repository execution for scheduler invariants, Expo linting, and PDF generation through the installed TeX toolchain.

2.8 Design and Implementation Constraints

- The system depends on valid Supabase environment variables and network connectivity.
- Most application features require an authenticated session.

- Image OCR depends on a native-capable development build and is not fully supported in Expo Go.
- Access to stored files is constrained by authenticated ownership and bucket policies.
- The scheduler is deterministic and rule-based; it is not a metaheuristic or search-optimization engine.
- Teacher-initiated block placements that must survive future rebalance depend on locked schedule state being preserved in storage.
- AI-assisted activity generation depends on configured backend function credentials and should be presented as teacher-editable draft generation rather than autonomous final assessment authoring.

2.9 Assumptions and Dependencies

- Teachers will define valid lesson-plan date ranges and recurring meeting patterns.
- Institutions will maintain accurate subject, section, and lesson content data.
- Curriculum extraction and activity generation depend on backend utilities and external processing services being available.
- The mobile user has permission to access local files or photos when uploading curriculum or template assets.
- The live defense environment will include prepared demo records so that schedule generation, rebalance, extraction, and activity-generation flows can be shown within the available presentation time.

3 External Interface Requirements

3.1 User Interfaces

The system is organized around authenticated mobile workflows. Major user-facing modules are shown in Figure 2.

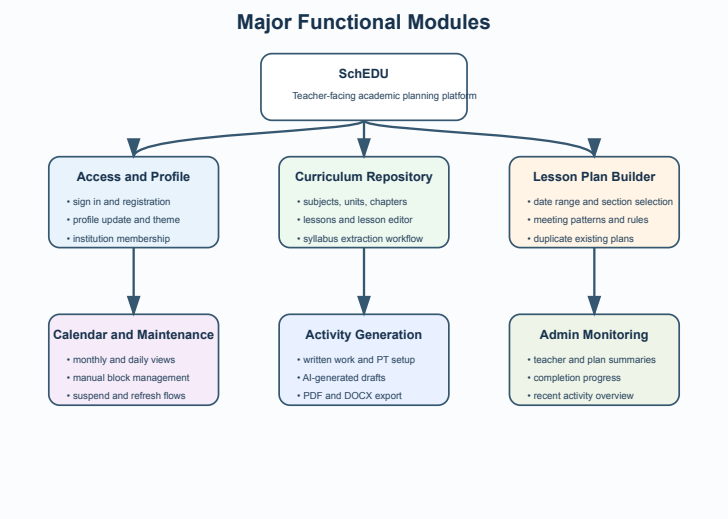


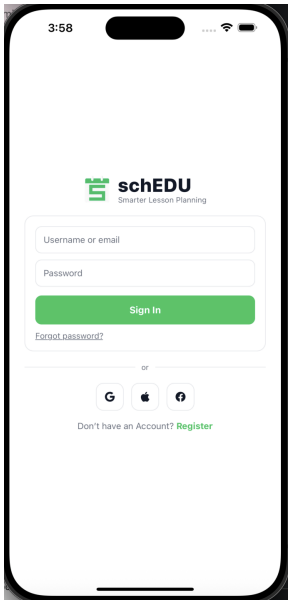
Figure 2: Major functional modules

The major interface groups are:

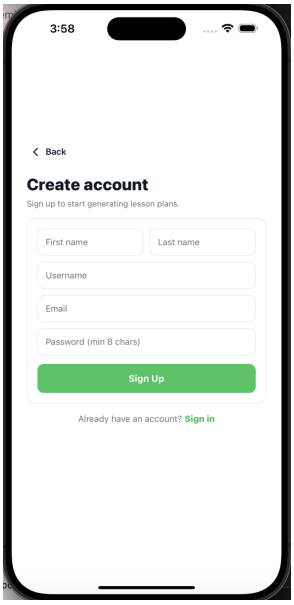
- **Authentication interfaces:** sign-in, sign-up, forgot-password, OAuth entry points, and password update screens.
- **Profile interfaces:** profile summary, settings, appearance theme selection, and institution detail screens.
- **Library interfaces:** subject list, subject detail, lesson detail, lesson editor, written-work detail, and performance-task detail screens.
- **Planning interfaces:** lesson-plan creation, plan list, and plan detail screens.
- **Calendar interfaces:** monthly plan calendar, daily agenda, block editor funnel, and suspension actions.
- **Activity interfaces:** form-driven activity creation, AI generation, preview, and export/share actions.
- **Administrative interfaces:** dashboard summaries for teachers, subjects, plans, and completion status.

3.1.1 Representative UI Screens

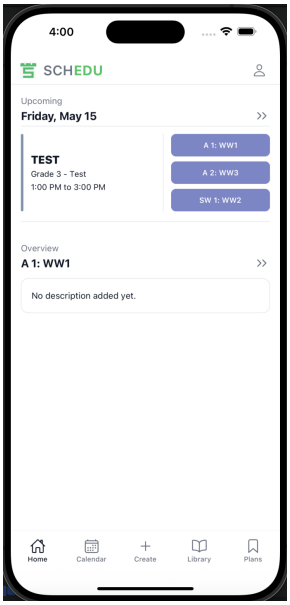
Figures 3 through 6 show representative screenshots from the current mobile application. These images document the implemented interface style and the major workflow surfaces that will be used in the live defense demonstration.



Login

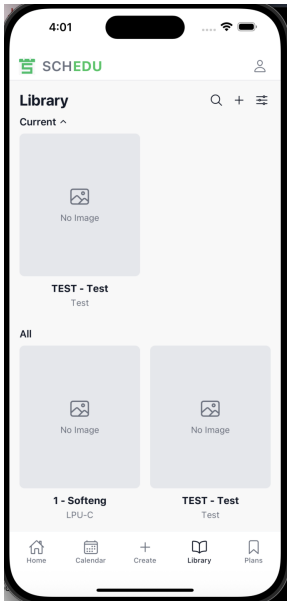


Register

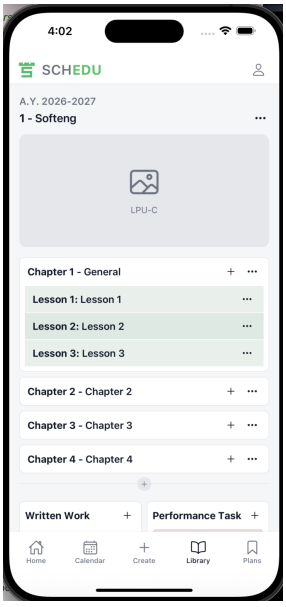


Dashboard

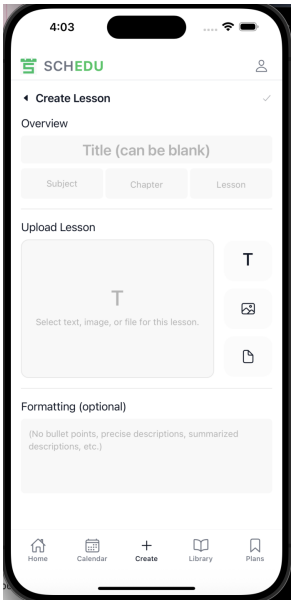
Figure 3: Authentication and dashboard screens



Library

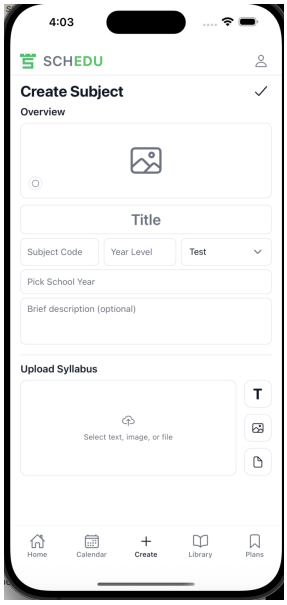


Subject Detail

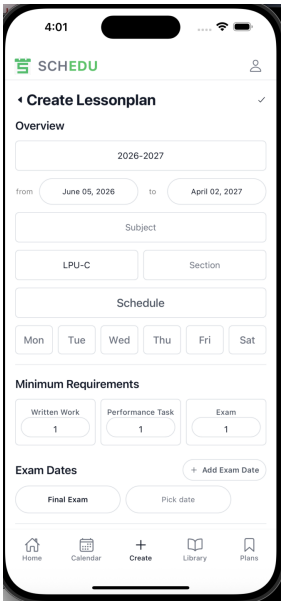


Lesson Detail

Figure 4: Curriculum library and lesson-content screens



Create Subject

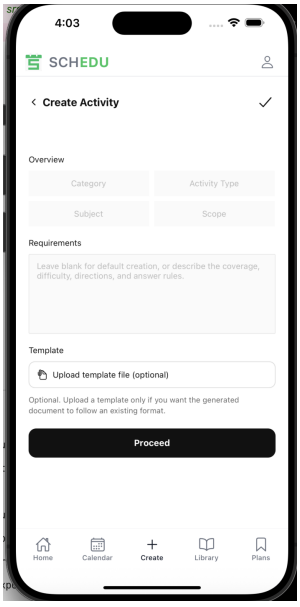


Create Lesson Plan

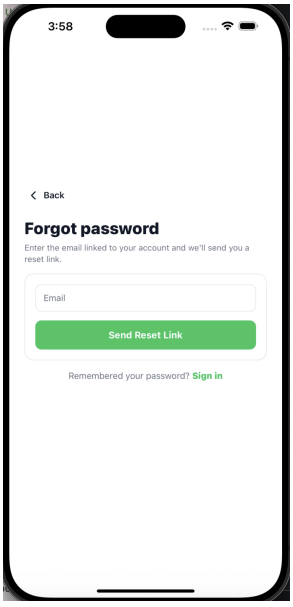
Calendar

Figure 5: Subject creation, lesson-plan creation, and calendar screens

Plans



Activity Generation



Alternate Register View

Figure 6: Plan management, activity-generation, and secondary account screen

3.2 Hardware Interfaces

The application is intended for smartphones and tablets capable of running Expo or native React Native builds. File upload features depend on local storage and, for image-based flows, camera roll access or equivalent media permissions.

3.3 Software Interfaces

- **Supabase Auth:** user sign-in, sign-up, session persistence, password reset, and OAuth integration.
- **Supabase PostgreSQL:** structured storage for users, schools, sections, subjects, units, chapters, lessons, lesson plans, slots, blocks, calendar events, delays, and activities.
- **Supabase Storage:** upload and retrieval of subject images, school avatars, PDFs, and generated activity artifacts.
- **Supabase Edge Functions:** `extract-text` and `generate-activity`.
- **OCR library:** image text extraction in supported native builds.
- **Document libraries:** PDF and DOCX generation for exported activity files.

3.4 Communication Interfaces

The client communicates with Supabase services and Edge Functions over HTTPS. Data exchange is primarily JSON-based for application records and multipart or binary upload flows for files stored in the application bucket.

4 Data Requirements

4.1 Core Data Entities

Entity	Description
users	Stores teacher and administrative identity, role, and profile information.
schools	Stores institution records including school type, avatar data, and creator metadata.
sections	Stores academic sections associated with schools.
subjects	Stores teacher-owned subject records and syllabus-related metadata.
units, chapters, lessons	Store the curriculum hierarchy and lesson content used for planning.
lesson_plans	Stores teacher lesson-plan context including subject, section, date range, and notes.
plan subject content	Stores the selected content scope used within a lesson plan.
slots	Stores generated or maintained schedule slots for a lesson plan.

blocks	Stores scheduled instructional and assessment units, including manual blocks.
school calendar events	Stores institution-level blackout events such as holidays, suspensions, and related blackout periods.
delays	Stores teacher- or plan-related delays and absence dates affecting planning.
activities	Stores AI-assisted written-work and performance-task drafts and associated artifacts.

4.2 Data Integrity and Business Rules

The system shall enforce the following data rules:

- lesson-plan end dates shall not be earlier than start dates;
- slot end time shall be later than slot start time;
- slot occurrences shall be unique per lesson plan, date, and slot number;
- block category and subcategory pairings shall match supported instructional types;
- activities shall belong only to written-work or performance-task categories;
- sequence numbering for units, chapters, and lessons shall preserve curriculum order within their parent scope;
- teacher-pinned schedule edits that must survive regeneration or rebalance shall be represented as locked slot or block state through the planner and calendar workflows; and
- user file access shall remain scoped to owned storage paths and authenticated access policies.

4.3 Data Lifecycle Requirements

SchEDU data moves through a defined lifecycle during normal use:

1. the authenticated user creates or joins an institution context;
2. the user creates sections, subjects, and subject content;
3. a lesson plan binds the subject, section, date range, meeting pattern, and selected content scope;
4. the scheduler derives slots and blocks from the saved planning inputs;
5. calendar workflows read, edit, suspend, restore, or rebalance the persisted plan state; and
6. activity and extraction workflows store generated text, file paths, and supporting metadata under the authenticated user context.

The system shall not treat generated schedule state as disposable UI-only data. Slots, blocks, selected content scope, calendar events, delays, and activity artifacts must remain inspectable through backend records so that a defense demo can show continuity between data entry, generated output, and later maintenance.

5 System Features and Functional Requirements

5.1 Authentication and Session Management

Description: This feature set manages account access, password-based and OAuth sign-in, recovery, and session routing.

Priority: High

ID	Requirement
FR-01	The system shall allow a user to register an account using email and password credentials.
FR-02	The system shall allow a user to sign in using email and password credentials.
FR-03	The system shall support username-based sign-in by resolving the stored username to the associated email address.
FR-04	The system shall support OAuth sign-in with the configured third-party providers.
FR-05	The system shall provide a forgot-password flow for account recovery.
FR-06	The system shall provide an update-password flow for authenticated password reset completion.
FR-07	The system shall persist authenticated sessions and redirect signed-in users away from authentication screens.
FR-08	The system shall redirect unauthenticated users away from protected application routes.
FR-09	The system shall allow a signed-in user to terminate the current session by signing out.

5.2 Profile, Theme, Institution, and Section Management

Description: This feature set maintains the user's profile, appearance settings, institution records, and sections.

Priority: High

ID	Requirement
FR-10	The system shall allow a signed-in user to view stored profile details including first name, last name, username, and email.
FR-11	The system shall allow a signed-in user to update profile details.

FR-12	The system shall allow a signed-in user to change the application appearance theme among system, light, and dark modes.
FR-13	The system shall display the institutions associated with the signed-in user.
FR-14	The system shall allow a user to create a new institution record with a name, school type, and visual identity settings.
FR-15	The system shall allow a user to upload and store an institution avatar image.
FR-16	The system shall allow a user to update institution information, including name, type, avatar image, and avatar color.
FR-17	The system shall allow a user to view all sections under a selected institution.
FR-18	The system shall allow a user to create a new section for a selected institution.
FR-19	The system shall allow a user to edit section details such as name and grade level.
FR-20	The system shall allow a user to delete a section subject to backend constraints.

5.3 Subject, Curriculum, and Lesson Management

Description: This feature set stores academic content that becomes the input to planning and scheduling workflows.

Priority: High

ID	Requirement
FR-21	The system shall allow a user to create a subject under a selected institution.
FR-22	The system shall allow a user to store subject metadata including code, title, year, academic year, description, and image.
FR-23	The system shall allow a user to provide syllabus or curriculum input as direct text, uploaded image, or uploaded file.
FR-24	The system shall upload syllabus-related assets to application storage under the authenticated user context.

FR-25	The system shall allow a user to invoke curriculum text extraction for uploaded PDF or image assets.
FR-26	The system shall allow a user to review and edit extracted or parsed curriculum text before saving subject content.
FR-27	The system shall store curriculum hierarchy as units, chapters, and lessons.
FR-28	The system shall allow a user to create lesson content manually without relying on imported curriculum text.
FR-29	The system shall allow a user to store and edit lesson titles, content, learning objectives, and estimated lesson information.
FR-30	The system shall display the authenticated user's subjects in a library view.
FR-31	The system shall support searching and filtering the subject library by query, institution, and year where data is available.
FR-32	The system shall allow a user to open a subject detail view showing related lesson and activity context.
FR-33	The system shall provide lesson detail and lesson-editor screens for detailed lesson review and editing.

5.4 Lesson Planning and Schedule Generation

Description: This feature set creates lesson plans and translates stored curriculum data into schedulable state.

Priority: High

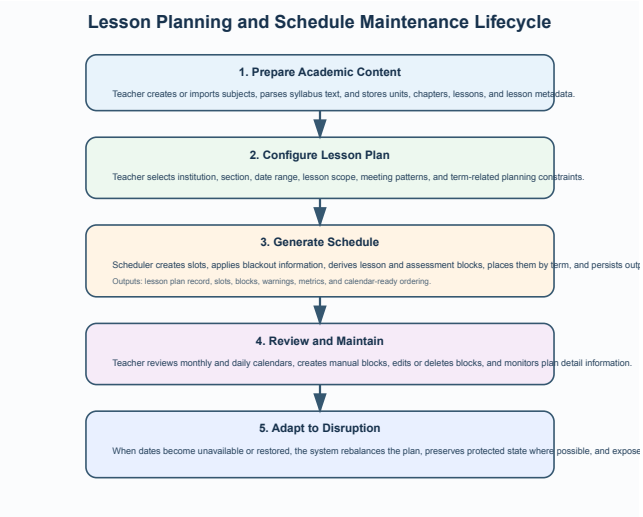


Figure 7: Lesson planning and schedule maintenance lifecycle

ID	Requirement
FR-34	The system shall allow a user to create a lesson plan for a selected institution, subject, and section.
FR-35	The system shall require the user to define a lesson-plan start date and end date.
FR-36	The system shall allow a user to define recurring meeting patterns including weekday, start time, end time, meeting type, and multiple class instances per day.
FR-37	The system shall allow a user to define lesson-plan scheduling constraints, including academic term structure, special dates, blackout dates, and exam-related dates collected by the planner flow.
FR-38	The system shall allow a user to select the lesson scope that will be included in the lesson plan.
FR-39	The system shall allow a user to duplicate an existing lesson plan into a new draft.
FR-40	The system shall validate lesson-plan input before generation or save.
FR-41	The system shall generate schedule slots by expanding recurring meeting patterns into dated occurrences with slot numbers, series keys, and normalized time ranges.
FR-42	The system shall apply blackout constraints from school calendar events and delays when generating or maintaining slots, and shall preserve compatible existing slot identifiers, lock flags, and manual state during regeneration.
FR-43	The system shall partition the lesson plan into academic terms anchored by exam dates and shall derive lesson, quiz, written-work, performance-task, exam, and buffer blocks with term metadata and dependency information.
FR-44	The system shall place generated blocks onto available slots using deterministic constraint-based greedy scheduling logic that reserves exam slots, assigns lesson blocks sequentially, and releases due assessments according to dependency and interval rules.
FR-45	The system shall persist lesson-plan records, selected content scope, slots, and blocks after successful generation.

FR-46	The system shall compute and expose schedule warnings and metrics for the generated plan, including total slots, usable slots, total blocks, placed blocks, unplaced blocks, category counts, and rebalance warnings.
FR-46A	The system shall rebalance a lesson plan deterministically after class-day availability changes by re-flowing non-locked blocks, preserving protected blocks, and surfacing locked-block conflicts instead of silently relocating teacher-pinned blocks.
FR-47	The system shall allow a user to view lesson plans in a dedicated plans list.
FR-48	The system shall allow a user to open a lesson-plan detail view showing metadata, status, and recurring schedule information.
FR-49	The system shall allow a user to edit lesson-plan metadata and schedule-related draft details from the plan detail workflow.

5.5 Calendar Viewing and Schedule Maintenance

Description: This feature set presents generated schedules and supports calendar-level maintenance operations.

Priority: High

ID	Requirement
FR-50	The system shall display a monthly calendar view for a selected lesson plan.
FR-51	The system shall allow the user to switch among available lesson plans in the calendar view.
FR-52	The system shall display a daily agenda view containing scheduled blocks across the user's lesson plans for a selected date.
FR-53	The system shall allow a user to select a day from the calendar and open its daily agenda.
FR-54	The system shall allow a user to create a manual block for a selected day.
FR-55	The system shall allow a user to edit an existing scheduled or manual block.
FR-56	The system shall allow a user to delete a block from the daily agenda workflow.

FR-57	The system shall allow a user to suspend or unsuspend selected lesson-plan entries from the daily agenda workflow using a recorded reason.
FR-58	The system shall refresh plan and calendar screens after relevant planning or block-edit operations.
FR-59	The system shall preserve teacher-locked or manually created schedule state during refresh and rebalance operations, unless the user explicitly edits or deletes that state.
FR-60	The system shall allow a user to navigate from a daily agenda block to the related lesson, written-work, or performance-task detail when such detail exists.

5.6 Activity Generation and Document Support

Description: This feature set helps teachers create assessment documents and supporting artifacts around planned instruction.

Priority: Medium to High

ID	Requirement
FR-61	The system shall allow a user to create an activity draft under either the written-work or performance-task category.
FR-62	The system shall allow a user to select an activity type appropriate to the selected category.
FR-63	The system shall allow a user to select a subject and lesson scope for the activity.
FR-64	The system shall allow a user to define activity requirements, selected components, and optional template guidance.
FR-65	The system shall allow a user to upload template assets used to guide activity generation.
FR-66	The system shall allow the user to invoke AI-assisted activity generation.
FR-67	The system shall return generated activity text in an editable form.
FR-68	The system shall allow the user to export generated activity output to PDF.
FR-69	The system shall allow the user to export generated activity output to DOCX.

FR-70	The system shall store generated activity metadata, selected scope, generated text, and artifact paths for later retrieval by the activity workflow.
FR-71	The system shall provide curriculum text extraction support for uploaded subject or lesson source documents.

5.7 Administrative Monitoring

Description: This feature set provides an overview of current teachers, plans, and planning progress for administrative users.

Priority: Medium

ID	Requirement
FR-72	The system shall provide an administrative dashboard for authorized users.
FR-73	The administrative dashboard shall display summary cards such as teacher count, subject count, lesson-plan count, and completion indicators from available backend records.
FR-74	The administrative dashboard shall display current or upcoming lesson-plan information.
FR-75	The administrative dashboard shall display teacher-related summaries such as names, roles, and plan counts from available backend records.
FR-76	The administrative dashboard shall display recent planning or activity indicators derived from available backend records.

6 Non-Functional Requirements

6.1 Performance Requirements

ID	Requirement
NFR-01	Under normal network conditions, the system should complete sign-in and initial route resolution within 5 seconds.
NFR-02	Under normal network conditions, list screens such as plans and library should display primary content or a recoverable error state within 5 seconds after refresh is initiated.
NFR-03	The system should complete schedule generation for a single lesson plan within 10 seconds for ordinary teacher planning workloads.

NFR-04	Long-running extraction or generation tasks shall provide visible progress or status feedback to the user.

6.2 Security and Privacy Requirements

ID	Requirement
NFR-05	The system shall use authenticated session management for protected features.
NFR-06	The system shall restrict user-owned database and storage data through backend access controls such as RLS and authenticated storage rules.
NFR-07	The system shall transmit client-backend traffic over HTTPS.
NFR-08	The system shall isolate user file uploads by authenticated ownership path.
NFR-09	The system shall be designed to support institutional privacy obligations, including proper handling of user account data and uploaded instructional documents.

6.3 Reliability and Data Integrity Requirements

ID	Requirement
NFR-10	The system shall preserve lesson-plan, slot, and block records unless explicitly deleted by an authorized workflow.
NFR-11	The system shall fail gracefully when backend or utility services are unavailable and shall provide an error message instead of silently discarding data.
NFR-12	The scheduling pipeline shall behave deterministically for the same input state, producing the same slot, block, and ordering outcomes when the inputs remain unchanged.
NFR-13	Rebalance operations shall preserve protected schedule state where the scheduler rules specify such behavior and shall behave idempotently for an unchanged schedule state.

6.4 Usability Requirements

ID	Requirement

NFR-14	The application shall present mobile-friendly layouts suitable for handheld devices.
NFR-15	The application shall provide understandable validation or error messages for failed form submissions and backend operations.
NFR-16	The application shall support pull-to-refresh or equivalent refresh actions on major data views.
NFR-17	The application shall support user-selectable appearance themes.

6.5 Maintainability and Portability Requirements

ID	Requirement
NFR-18	The system shall maintain modular separation between UI flows, backend access helpers, and scheduling logic.
NFR-19	The codebase shall remain TypeScript-based and lintable in the existing project toolchain.
NFR-20	The primary production targets shall remain Android and iOS devices supported by the Expo/React Native stack.
NFR-21	Utility features that require native capabilities shall degrade gracefully or document their runtime limitations.
NFR-22	The defense build shall support a prepared live demonstration of authentication, academic setup, lesson-plan generation, calendar maintenance, and at least one document-support workflow.
NFR-23	The project documentation shall distinguish implemented deterministic scheduling behavior from unsupported optimization claims.

7 Acceptance Criteria and Traceability

7.1 Defense Acceptance Criteria

The system shall be considered defense-ready when the following criteria are met:

ID	Acceptance Criterion	Verification Evidence

AC-01	A user can authenticate and reach protected teacher workflows.	Auth screens, protected route redirection, Supabase Auth session checks, and live sign-in demonstration.
AC-02	A teacher can create or load institution, section, subject, chapter, and lesson data.	Profile, institution, library, subject-detail, lesson-detail, and lesson-editor screens backed by academic tables.
AC-03	A teacher can create a lesson plan from selected subject content, date range, meeting patterns, and term settings.	Lesson-plan creation flow persists lesson plan, selected content, generated slots, and generated blocks.
AC-04	The scheduler can generate a deterministic plan with placed lessons, assessments, exam anchors, metrics, and warnings.	Repository scheduler modules and passing `npm run test:scheduler` invariant execution.
AC-05	A suspended or restored teaching day triggers deterministic rebalance without losing blocks.	Invariant harness covers suspend, unsuspend, compression, restoration, and idempotency scenarios.
AC-06	A teacher-locked block that becomes infeasible is surfaced as a visible conflict rather than silently moved.	Invariant harness covers locked-block displacement, conflict warning, suggested re-pin slots, and re-pin recovery.
AC-07	Calendar views display plan output and allow day-level inspection or maintenance.	Calendar monthly and daily screens, block editor workflow, refresh behavior, and live demo walkthrough.
AC-08	Curriculum extraction and activity generation are guarded by authenticated backend calls.	Supabase Storage path ownership, `extract-text` Edge Function authentication, `generate-activity` validation, and prepared utility demo.

AC-09	Administrative monitoring renders summary information for authorized users.	Admin route group and dashboard cards for teachers, subjects, plans, and recent activity.
AC-10	Documentation and paper claims remain aligned with implementation boundaries.	This SRS, IEEE paper, scheduler invariant output, lint output, and explicit boundary section.

7.2 Requirements Traceability Matrix

Req. Group	**Implementation Area**	**Primary Data or Service**	**Validation Evidence**
FR-01–FR-09	Authentication route group and root layout guards	Supabase Auth and user records	Live sign-in, sign-out, password recovery, route protection, and lint validation.
FR-10–FR-20	Profile, settings, institution, and section screens	`users`, `schools`, `sections`, membership tables, storage avatars	Repository-backed workflow tracing and live profile/institution demo.
FR-21–FR-33	Library, subject, lesson detail, and lesson editor screens	`subjects`, `units`, `chapters`, `lessons`, uploaded syllabus assets	Live subject/library demo and schema-backed persistence.
FR-34–FR-46A	Lesson-plan creation and scheduling engine	`lesson_plans`, selected content, `slots`, `blocks`, scheduler modules	Passing scheduler invariants, generated metrics, and live lesson-plan generation.
FR-47–FR-60	Plans list, plan detail, monthly calendar, daily agenda, block editor, and suspension actions	Persisted plan, slot, block, calendar event, and delay records	Live calendar walkthrough and rebalance invariant coverage.

FR-61–FR-71	Activity creation, curriculum extraction, PDF/DOCX export, and sharing utilities	`activities`, Supabase Storage, `extract-text`, `generate-activity`	Edge Function guard inspection and prepared utility demo.
FR-72–FR-76	Administrative route group and dashboard	User, subject, lesson-plan, activity, and completion summaries	Repository-backed admin screen tracing and authorized-role demo.
NFR-01–NFR-04	App loading, refresh, scheduling, and long-running utility feedback	Expo runtime, Supabase network calls, scheduler pipeline	Live timing observation and visible loading/error states.
NFR-05–NFR-09	Authentication, RLS, HTTPS, storage ownership, and privacy controls	Supabase Auth, PostgreSQL RLS, Storage policies, Edge Function bearer tokens	Schema and Edge Function inspection plus authenticated demo flow.
NFR-10–NFR-23	Reliability, determinism, usability, maintainability, portability, demo readiness, and documentation accuracy	TypeScript modules, Expo screens, LaTeX documents, validation scripts	`npm run test:scheduler`, `npm run lint`, PDF rebuilds, and explicit limitations.

7.3 Validation Status for Defense Package

The current validation package is mixed but defensible for a live academic defense. Scheduler behavior is directly validated by executable invariants. Code style is checked by Expo linting, which currently reports no errors and known warnings. Broader mobile workflows are validated by repository-backed tracing and should be complemented with a live device walkthrough during defense. Utility flows depend on configured Supabase functions and external service credentials, so backup prepared records or screenshots should be available if the network or AI service is unavailable during the presentation.

8 Current System Boundaries

The current version of SchEDU does **not** claim the following as supported core features:

- institution-wide room allocation or campus-wide timetable optimization;
- automated grading of student submissions;
- full learning management system synchronization;
- formal search-based optimization methods such as constraint programming or simulated annealing; and
- unrestricted offline operation for backend-dependent workflows.

These boundaries are intentional in this SRS so that the specification remains aligned with the implemented repository and the current product scope.